PROMPTER

Full Tier-Wise Plan: Idea → AI Engine → Web App → Production

Vision: A user describes what they want to accomplish. Prompter understands the user and task, finds suitable AI tools, explains the fit, generates a tool-specific prompt, and opens the selected AI.

BUILD PRINCIPLE

Build one complete working loop first. Do not start by building a huge AI directory, complex multi-agent system, or mobile app.

V1 loop: User Profile + Problem → Task Understanding → AI Tool Matching → Why → Tool-Specific Prompt → Copy/Open AI

1. PROMPTER V1 — WHAT WE ARE BUILDING

Prompter is an AI discovery and routing layer, not merely a prompt generator. The user should not need to know which AI tool to search for or how to formulate the request.

```
USER
↓
Text / Voice
↓
User Context + Task Understanding
↓
AI Tool Matching
↓
Recommendation + Why
↓
Tool-Specific Prompt
↓
Copy Prompt / Open AI
```

Core product inputs

- User role: student, professional, researcher, developer, entrepreneur, etc.
- Domain/field, skills, interests, goals and relevant preferences.
- Current problem/task, including inputs, desired output, constraints and budget preference.
- Optional voice and file inputs after the text workflow works.

Core product output

- Structured understanding of the user's task.
- 1–3 suitable AI-tool options with factual reasons.
- A prompt adapted to the selected tool and task.
- Copy prompt and Open AI actions.
- Feedback for improving future recommendations.

2. MASTER TIER ROADMAP

Tier	Focus	Deliverable
0	Foundation	Scope, requirements, GitHub, architecture
1	Data model	UserProfile, TaskProfile, AITool schemas
2	AI brain	Natural language → structured task
3	Tool knowledge	Initial 30–50 AI tools
4	Matching	Suitable tools + reasons
5	Prompt engine	Tool-specific optimized prompts
6	Backend	FastAPI + PostgreSQL APIs
7	Frontend	Next.js/React web application
8	Personalization	Login + onboarding + profile
9	Voice/files	Speech + document inputs
10	Evaluation	Feedback + analytics + test set
11	Production	Deployment + security + monitoring
12	V2 intelligence	RAG + embeddings + retrieval
13	Future	Integrations, agents, mobile

TIER 0 PRODUCT FOUNDATION

Freeze the MVP before coding.

Tasks

- Write the exact V1 journey: signup → profile → problem → recommendation → prompt → open AI.
- Define out-of-scope features: autonomous execution, huge catalog, payments, social features and mobile-first.
- Create GitHub repository, README and issue board.
- Choose stack: Next.js/React, Tailwind, FastAPI/Python, PostgreSQL and an LLM API.
- Create .env.example and keep all secrets outside Git.

Deliverable: product specification + architecture + working repository.

TIER 1 PROMPTER DATA MODEL

Define what the system knows about users, tasks and tools.

UserProfile

```
{
"role": "student",
"education_level": "undergraduate",
"field": "electrical_engineering",
"skills": ["Python","Machine Learning"],
"interests": ["AI","Data Science"],
"goals": ["projects","internship"],
"preferred_language": "English"
}
```

TaskProfile

```
{
"domain": "academic",
"task": "create_presentation",
"input": ["PDF"],
"output": "PPT",
"complexity": "medium",
"needs_web": false,
"needs_data_analysis": false,
"needs_image_generation": true,
"budget": "free"
}
```

Also define: AITool, Recommendation, GeneratedPrompt and Feedback schemas.

TIER 2 AI INTELLIGENCE ENGINE

Convert natural language into structured task information.

- Connect an LLM API.
- Request structured JSON output rather than free-form analysis.
- Extract domain, task, inputs, desired output, constraints, budget, urgency and language.
- Validate the result with a schema and handle uncertainty/failures.
- Create 20–30 test prompts representing different real user tasks.

Example

```
User: "I have a research paper and tomorrow I need
a professional college presentation."

→ domain: academic
→ task: create_presentation
```

```
→ input: research paper/PDF
→ output: presentation
→ urgency: high
```

Milestone: reliable structured task extraction across representative examples.

TIER 3 AI TOOL KNOWLEDGE BASE

Start small and maintain quality.

Begin with approximately 30–50 carefully selected AI tools. Do not start with 1,000+ tools.

```
{
"name": "...",
"category": "...",
"capabilities": [],
"input_types": [],
"output_types": [],
"best_for": [],
"limitations": [],
"languages": [],
"website": "...",
"pricing": "...",
"prompt_guidance": "...",
"last_verified": "YYYY-MM-DD"
}
```

- Organize tools by capability: writing, coding, research, data analysis, presentation, image, video, design, education, business, etc.
- Store verification dates because tool features and pricing/free limits can change.
- Keep tool descriptions factual and update them systematically.

TIER 4 RECOMMENDATION ENGINE

Match user context + task against tool capabilities.

Start with transparent rule/score matching; upgrade later with embeddings and learning from feedback.

```
score =
capability_match
+ input_match
+ output_match
+ domain_fit
+ user_context_fit
+ constraint_fit
+ preference_fit
```

- Return 1–3 suitable options instead of claiming there is always one universal 'best' AI.
- Show why each option fits the task.
- Store recommendation and feedback data for later improvement.

Milestone: structured task → suitable tool candidates + reasons.

TIER 5 TOOL-SPECIFIC PROMPT ENGINE

Turn the task into a useful prompt for the selected tool.

Combine relevant user context, task requirements, input/source, desired output, quality criteria, constraints and the selected tool's capabilities.

```
ROLE / CONTEXT
TASK
INPUT / SOURCE
REQUIREMENTS
OUTPUT FORMAT
QUALITY CRITERIA
CONSTRAINTS
FINAL CHECKS
```

Important: only include profile information when it improves the task. Do not unnecessarily send personal information to external tools.

Milestone: selected tool + task → clean, tool-specific prompt ready to copy.

TIER 6 BACKEND API

Turn the intelligence engine into a service.

Technology: Python + FastAPI + PostgreSQL.

```
POST /analyze-task
POST /recommend-tools
POST /generate-prompt

GET /tools
GET /tools/{id}

POST /feedback
Request
↓
User ID / authentication
↓
Load user profile
↓
Analyze task
↓
Match tools
↓
Generate explanation
↓
Generate prompt
↓
Return structured response
```

- Use Pydantic validation.
- Handle errors and timeouts.
- Add logging and rate limiting before public release.

TIER 7 FRONTEND WEB APP

Build the first real Prompter interface.

Technology: Next.js + React + Tailwind CSS.

- Landing page
- Sign up/Login
- Onboarding/Profile
- Dashboard
- Task input
- Recommendation result
- Generated prompt
- History

```
Your task
■■■■■■■■■■
[understood task]

Suitable AI tools
■■■■■■■■■■■■■■■■■■
Tool A — why it fits
Tool B — why it fits

Generated prompt
■■■■■■■■■■■■■■■■
[ prompt ]
```

Build web first. Validate the workflow before investing in mobile.

TIER 8 AUTHENTICATION & PERSONALIZATION

Make Prompter aware of who the user is.

Keep onboarding short and task-relevant.

- Role
- Field/industry
- Skills and experience level
- Interests/common tasks
- Goals
- Preferred language/output preferences

```
User Profile
+
Current Task
+
Preferences
↓
```

Privacy: collect only useful information, give users control over their profile, and avoid passing unnecessary profile data to external AI services.

TIER 9 VOICE & FILE INPUTS

Add richer input after text is stable.

```
VOICE → Speech-to-text → Task understanding
→ Recommendation → Prompt

PDF / Document / Dataset
→ Relevant extraction
→ Task understanding
→ Tool matching
```

Text first. Voice is simply another input layer once the core intelligence works.

TIER 10 FEEDBACK & EVALUATION

Measure whether recommendations actually help.

- Was the recommendation useful?
- Did the prompt work?
- Did the user copy/open the recommended tool?
- Where does task classification fail?
- Which tools are repeatedly selected?
- Measure latency and AI/API cost

Create a fixed evaluation set of representative tasks and run it whenever matching or prompting logic changes.

TIER 11 PRODUCTION DEPLOYMENT

Make the web app reliable and accessible.

```
Browser
↓
Next.js frontend
↓
FastAPI backend
↓
PostgreSQL
↓
LLM / speech / external AI links
```

- Environment variables and secret management.
- HTTPS, database backups and access control.
- Rate limiting and abuse protection.
- Monitoring and error tracking.
- Input/output validation.
- Tool-information verification process.
- Basic privacy/terms documentation appropriate to the service.

V1 launch target: account → profile → describe problem → suitable AI options + reasons → generated prompt → copy/open AI.

TIER 12 V2 INTELLIGENCE: RAG + EMBEDDINGS

Add retrieval when the tool catalog becomes large or dynamic.

```
AI tool documents
↓
Metadata + chunks
↓
Embeddings / vector search
↓
Relevant capabilities/tools
↓
LLM recommendation
```

- Use retrieval for changing tool capabilities, documentation, limitations and verified information.
- Keep metadata such as category, input/output, language and last verification.
- Do not add vector infrastructure merely because it is fashionable; add it when retrieval solves a real problem.

TIER 13 FUTURE PLATFORM

Only after V1/V2 has real usage and feedback.

- Direct API integrations with selected AI tools.
- One-click execution where APIs permit it.
- Multi-step workflows and agent orchestration.
- Long-term user preferences with clear controls.
- Mobile application.
- Team/workspace and enterprise workflows.
- Learning-to-rank recommendations from product feedback.

```
V1: Understand → Recommend → Explain → Prompt → Open

V2: Understand → Retrieve → Recommend → Prompt → Open

Future:
Understand → Plan → Select tools → Execute
→ Verify → Return outcome
```

3. WHAT YOU NEED TO LEARN

Now: FastAPI, REST APIs, Pydantic, PostgreSQL/SQL, LLM APIs, structured outputs.
Next: React/Next.js, authentication, deployment, testing.
Later: embeddings, RAG, agents, integrations and mobile.

4. PROJECT STRUCTURE

```
prompter/
■■■ backend/
■ ■■■ main.py
■ ■■■ api/
■ ■■■ models/
■ ■■■ schemas/
■ ■■■ services/
■ ■ ■■■ task_analyzer.py
■ ■ ■■■ matcher.py
■ ■ ■■■ prompt_generator.py
■ ■■■ database/
■■■ data/
■ ■■■ ai_tools.json
■■■ frontend/
■ ■■■ next-app/
■■■ tests/
■■■ .env.example
■■■ requirements.txt
■■■ README.md
```

5. FIRST WORKING VERTICAL SLICE

Make one example work from start to finish before generalizing.

```
Input:
"I have a research paper and tomorrow I need a
professional college presentation."

Output:
Domain: Academic
Task: Presentation generation
Input: PDF
Output: PPT
User: Undergraduate
Constraint: Free/low-cost preference

→ Suitable tools
→ Reasons
→ Tool-specific prompt
→ Copy / Open
```

Then expand into coding, data analysis, research, writing, presentations, resume/interview work, image/video, business and education.

6. 30-DAY BUILD SEQUENCE

Days	Build
1–2	Repository, requirements, architecture, environment
3–5	UserProfile, TaskProfile, AITool schemas
6–8	LLM task analysis + structured JSON
9–11	Initial 10–30 tool dataset
12–14	Recommendation engine
15–16	Recommendation explanations
17–18	Prompt generation
19–21	FastAPI + PostgreSQL
22–24	Next.js UI + result screens
25–26	Authentication + onboarding
27	Copy/Open AI workflow
28	Feedback + analytics
29	Testing + bug fixes
30	Deploy MVP + document project

7. V1 DEFINITION OF DONE

- User can create an account and lightweight profile.
- User can describe a problem in natural language.
- Prompter converts it into structured task information.
- Prompter uses relevant user context.
- Prompter matches against a maintained AI-tool knowledge base.
- Prompter returns suitable options with factual reasons.
- Prompter generates a tool-specific prompt.
- User can copy the prompt and open the external tool.
- Feedback and basic analytics are stored.
- The web app is deployed and usable.

8. DO NOT BUILD YET

- 1,000+ tools.
- Complex autonomous multi-agent execution.
- Your own foundation model.
- Mobile-first app.
- Payments before validation.
- Large RAG infrastructure before it is needed.
- Huge social/community features.

9. IMMEDIATE NEXT STEPS

Start with the Prompter Intelligence Engine v0.1. The first coding target is the backend/data foundation, not UI polish.

```
1. Create GitHub repository
2. Set up Python + FastAPI
3. Define UserProfile
4. Define TaskProfile
5. Define AITool
6. Add first 10 AI tools
7. Connect LLM API
8. User sentence → TaskProfile
9. TaskProfile → suitable tools
10. Generate explanation + prompt
11. Test with 20–30 examples
12. Build frontend around the working engine
```

FINAL TARGET

Tell Prompter what you want to accomplish. Prompter understands your context, finds suitable AI tools, explains the fit, prepares the prompt, and gets you to the right tool.

Build intelligence first → interface second → validate the complete loop → then add RAG, voice, integrations, agents and mobile.